

1. The core mental model

Before roles and features, one structural decision shapes everything else: **model the health system as a tree (or graph) of facilities with parent-child replenishment relationships, and let shortage signals flow both up and down that tree.**

State Warehouse (1)

└─ District Hospital / District Warehouse (many)

└─ Block-level Hospital / CHC (many per district)

└─ PHC / Sub-center / Village Health Center (many per block)

Two things flow through this tree:

- **Supply flows downstream** (warehouse → district → block → village), triggered by requisitions/replenishment.
- **Signals flow upstream** (village → block → district → state): stock levels, consumption rates, and stockout risk.

The key insight your problem statement is pointing at: **a single empty shelf is a local event, but five empty shelves for the same medicine across one district, appearing within the same week, is a different kind of event** — that's the "early signal of wider disruption" the problem wants you to detect. So your system needs two distinct outputs, not one:

1. **Facility-level alert**: "PHC Rampur will run out of Oral Rehydration Salts in ~4 days."
2. **Regional-level signal**: "6 of 9 facilities in Block Chandpur show declining ORS stock trend with no matching replenishment — possible district-level supply disruption, not just local mismanagement."

That distinction is the actual novelty in this problem statement — most people will build #1 (an inventory tracker) and miss #2 (the epidemiology-of-shortages layer). I'd make #2 your differentiator in the pitch.

2. Users and what each one actually does

Let's go role by role, concretely — what screen they open, what they do on it, why they care.

A. Facility Admin (PHC / village health center / block CHC / district store)

This is the person physically standing near the shelf.

What they do:

- Log incoming stock (received from upstream, with batch no. + expiry + quantity)
- Log outgoing stock (dispensed to patients, or transferred out) – ideally in bulk/periodic entry, not per-patient, since this is a hackathon not an EMR
- View their own current inventory with status colors (Green/Yellow/Red/Critical per medicine)
- See a **forecast**: "At current consumption, Amoxicillin runs out in 6 days"
- Raise a **replenishment request** upstream (auto-suggested quantity, editable)
- Receive/accept an **incoming transfer** from a peer facility (redistribution)
- Get notified when their own data hasn't been updated in N days (data-quality nudge)

Why this helps them: Right now this person is probably tracking stock on paper or a register, and requesting replenishment reactively – often *after* they've already run out. Giving them a forecast turns their job from reactive to proactive, and gives them a paper trail for accountability.

B. Block/District Program Manager (mid-tier – oversees ~10-50 facilities)

This is the person who currently only finds out about a shortage when a facility calls them.

What they do:

- View a **map of their jurisdiction** with facilities color-coded by worst-medicine-status
- Select a medicine → see availability across all their facilities on the map
- See the **regional shortage signal** when it fires ("cluster of 6 facilities trending toward stockout")
- Approve/reject facility replenishment requests, or approve **redistribution suggestions** the system proposes (e.g., "Facility A has 400 surplus units expiring in 20 days; Facility B needs 150 units in 4 days; distance 12km – recommend transfer")
- Escalate to the state level if the shortage looks systemic rather than local (e.g., supplier delay, not local overconsumption)
- View a simple report of who is under-reporting (data reliability score per facility)

Why this helps them: They currently operate blind between facility phone calls. This gives them a live dashboard instead of a reactive one, and – critically – it tells them *why* a shortage is happening (consumption spike? delayed replenishment? bad reporting?) not just *that* it's happening.

C. State/System Admin

This is the super-admin and also, in your hackathon build, the person who manages master data.

What they do:

- Add/edit facilities, their hierarchy, and their geo-coordinates

- Add/edit the medicine catalog (name, category, unit, essential-medicine flag, shelf life)
- Set system-wide thresholds (what counts as "low stock" — could be a fixed days-of-stock number, or per-medicine)
- View the **state-wide map**, select any medicine, see availability heat-map
- View system-wide regional-shortage alerts across all districts, ranked by severity/confidence
- Drill down from a state-level alert into the underlying district → block → facility data that triggered it
- Generate a report for policymakers (e.g., "Top 5 medicines at systemic risk this month")

Why this helps them: This is the person the problem statement is really written for — "health authorities have enough time to respond." Their whole value-add is early warning + prioritization: which of the 50 simultaneous local problems is actually the one regional disruption that needs an urgent intervention (emergency procurement, supplier escalation, inter-district transfer)?

D. (Optional, adds polish) Policy/Decision-maker viewer

Read-only. Sees dashboards and trend reports only, no data entry. Useful if you want to show "different personas, different views" in your demo without extra workflow complexity.

3. The core workflows, in detail

Workflow 1 — Daily/periodic stock update (Facility Admin)

1. Admin logs in, sees their facility's medicine list
2. For each medicine: current stock, quick-entry for "dispensed since last update" and/or "received since last update"
3. System recomputes: current stock, consumption rate (rolling average), days-of-stock-remaining
4. System updates the medicine's status color and, if it crosses threshold, fires a facility-level alert

Workflow 2 — Replenishment request (Facility Admin → Block/District)

1. Facility admin sees Yellow/Red status on a medicine
2. Clicks "Request replenishment" — system pre-fills a suggested quantity (e.g., based on average monthly consumption minus current stock, or a fixed reorder-up-to level)
3. Request goes to the parent facility's queue
4. Parent facility (or system admin) approves, marks as dispatched, facility admin marks as received when it arrives — this closes the loop and also gives you **replenishment delay** as a measurable data point (this matters a lot for your forecasting, see below)

Workflow 3 — Shortage detection (system, automatic)

Run this per facility, per medicine, on every data update or on a schedule:

- **Days-of-stock-remaining** = current stock ÷ average daily consumption (rolling window, e.g. 7 or 14 days)
- Compare against thresholds → status: Green (>14 days), Yellow (7–14), Red (2–7), Critical (<2)
- Also track **trend**, not just snapshot: is days-of-stock-remaining shrinking week over week? A facility at "10 days remaining but stable" is less urgent than one at "10 days remaining and dropping fast."
- Also track **replenishment reliability**: average delay between request and receipt for that facility/medicine — a facility with a history of slow replenishment needs a bigger safety buffer, which you can factor into the forecast.

Workflow 4 – Regional signal detection (system, automatic – this is your key differentiator)

This runs at the block/district level, not facility level:

- For a given medicine, count how many facilities in a geographic/administrative cluster are Yellow/Red/Critical *simultaneously*
- Compare against a baseline (is this normal seasonal variation, or an anomaly?)
- Check whether it's correlated with a common cause: e.g., all facilities' last replenishment came from the same upstream warehouse, and that warehouse itself is showing declining stock — this is the tell-tale sign of an upstream supply problem rather than N independent local mismanagement problems
- If the pattern crosses a cluster threshold (e.g., "≥50% of facilities in this block are Red for Medicine X within a 7-day window"), fire a **regional shortage signal**, distinct from individual facility alerts, and escalate it to the district/state dashboard

Workflow 5 – Redistribution matching (system suggests, human approves)

1. System identifies a facility with a deficit (Red/Critical) for medicine X
2. System searches nearby facilities (same block/district, within a radius) for surplus of medicine X (defined as: high days-of-stock-remaining relative to their own consumption, *and* stock that isn't needed for their own buffer)
3. System ranks candidate matches by: distance, transport feasibility, expiry date (prefer to move stock closer to expiry — reduces wastage too), quantity match
4. Presents as a suggestion to the district manager: "Transfer 200 units from Facility A to Facility B — 12km, ~40 min drive, resolves Facility B's stockout risk"
5. Manager approves → becomes a transfer record, tracked like the replenishment workflow (dispatched → received)

Workflow 6 – Uncertainty communication

This is explicitly called out in the problem statement, so don't skip it. A few concrete ways to do this well, in increasing sophistication:

- **Simplest:** show a range, not a point estimate — "stockout expected in 4–7 days" instead of "4 days," derived from variance in the consumption history
- **Data quality flag:** if a facility hasn't updated in a while, show "estimate based on stale data (last updated 9 days ago)" — this is honest and easy to implement
- **Confidence tag:** Low/Medium/High confidence per alert, based on how much consumption history you have and how volatile it's been
- For a hackathon, I'd implement the range + staleness flag — that alone demonstrates the concept without needing real statistical modeling

4. Data model (minimum viable)

- **Facility:** id, name, type (village/block/district/state), parent_facility_id, lat, lng
- **Medicine:** id, name, category, unit, is_essential
- **InventoryRecord:** facility_id, medicine_id, current_stock, last_updated
- **ConsumptionLog:** facility_id, medicine_id, date, quantity_dispensed
- **ReplenishmentRequest:** from_facility_id, to_facility_id, medicine_id, quantity_requested, status (requested/approved/dispatched/received), timestamps for each status
- **TransferRecord:** same shape as replenishment, but facility-to-facility instead of up the hierarchy, plus a flag that it originated from a system suggestion
- **Alert:** type (facility/regional), medicine_id, facility_id or cluster_id, severity, confidence, created_at, resolved_at

That's genuinely enough to build every workflow above.

5. Feasibility for a hackathon build

- **Data:** don't try to source real hospital data — simulate it. Generate a synthetic set of ~30–50 facilities across 3–4 districts with lat/lng (use real Indian district/block names for realism), a catalog of ~15–20 essential medicines, and a consumption simulator that lets you *inject* a shortage scenario on demand for your demo (e.g., "simulate a supplier delay affecting District X for Medicine Y" so your demo can visibly show the regional-signal detection firing).
- **Stack:** a web dashboard (React) + a backend with a simple scheduled job (or just recompute on read) for the forecasting logic + a map library (Leaflet/Mapbox) for the geographic view. None of this needs ML — rolling averages and threshold rules are legitimate and explainable, which is actually a strength when you're pitching to a health-authority audience who will distrust a black box.

- **What to cut for time:** patient-level dispensing, batch/expiry-level tracking (keep expiry only for redistribution ranking, not full batch management), authentication complexity, real-time transport routing (fake the distance/time estimate).
- **What to keep no matter what:** the facility-level alert, the regional-signal detection, the map view with medicine-selector, and the redistribution suggestion — these four are the ones that actually answer the problem statement.

6. Business model (lightweight, since this is a govt-facing tool)

This isn't really a consumer SaaS, so keep the pitch realistic:

- **Primary model:** licensed to state health departments / procured via government e-marketplace (GeM in India), similar to how e-Aushadhi or IHIP-type systems are deployed
- **Secondary:** NGO/donor-funded deployment in specific districts as a pilot, with state government as eventual owner
- **Monetization for the hackathon pitch:** don't oversell this — judges will find "we'll charge governments a subscription" more credible than a consumer monetization story. Emphasize cost savings (reduced wastage from expiry, avoided emergency procurement) as the ROI case instead of a revenue model.